

Part 1

1. T时刻, PA、PB、PC 静态优先数均为100, p-cpu均为0

$$HZ=60, SCHMAG=20$$

$$p-pri = 100 + p-cpu/16$$

p-cpu:

	T	T+1	T+2	T+3	T+4
PA	0	40	20	0	40
PB	0	0	40	20	0
PC	0	0	0	40	20

	T	T+1	T+2	T+3	T+4
运行进程	PA	PB	PC	PA	PB
PA	100	102	101	100	102
PB	100	100	102	101	100
PC	100	100	100	102	101

2. T+3时刻 PA重新获得运行机会 (T+3时, PA的p-pri恢复至100)

保护现场: ①硬件自动压栈, 将SS、ESP等压入PA核心栈

②中断入口保存, 将通用寄存器压入PA核心栈

③进程切换保存: Switch()将核心栈指针ESP压入PA的user

如何恢复: ①Switch()从PA的user读核心栈指针ESP加载到CPU

②恢复通用寄存器

③IRET, 硬件自动弹出SS、EIP等, 将CPU切换回用户态

Part 2

(1) 系统调度: 时钟中断处理发现 $\text{Time::time}(\text{lo}) == \text{Time::time}(\text{tout})$
系统调用 $\text{WakeUpAll}(\&\text{tout})$, PA、PB 被唤醒, 变成 SRUN
PB 从 Sleep 返回, 退出系统调用返回用户态

(2) Sleep 系统调用下半部:

PA 执行 sys_sleep , 检查 $\text{wakeTime} > \text{Time::time}$ 成立,
更新 tout 为 1365, 再次调用 Sleep 让本 CPU, 变为 SSLEEP

(3) 直到 1365s, PA 检查 $\text{wakeTime} > \text{Time::time}$ 为假, 才会返回用户态
= PA 执行 Sleep, 用了 365 秒返回

期末总复习:

1. (1) 用户态现场

保存: ① 硬件自动压栈, 将 SS-ESP 等压入核心栈

② 中断入口保存, 将通用寄存器压入核心栈

恢复: ① 恢复通用寄存器

② IRET, 硬件自动弹出 SS-ESP 等, 将 CPU 切换回用户态

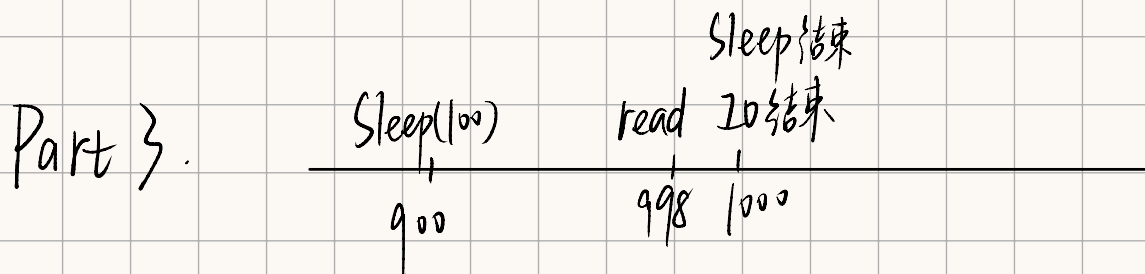
(2) 进程切换: $\text{Swch}()$

进程切换保存: $\text{Swch}()$ 将核心栈指针 ESP 压入 PA 的 user

进程切换恢复: $\text{Swch}()$ 从 user 读核心栈指针 ESP 加载到 CPU

2. 并发的系统调用:

PA和PB实际上是交替使用CPU, 执行Sleep系统调用, 通过全局变量tout和唤醒机制, 实现逻辑上的并发计时.



1. 磁盘中断无响应:

- ① 中断发生: 保存PX上下文(上面分析过), 跳转到磁盘中断入口程序
- ② 磁盘中断刚开始执行, 时钟中断到达, CPU暂停磁盘中断, 将现场压入核心栈, 跳转到时钟中断入口程序
- ③ 时钟中断执行, 检查到 $tout == 1000$, 将PA放入就绪队列
- ④ 时钟中断返回, 先前为核心理态, 将所有事务延迟至下一次时钟中断
- ⑤ 继续执行磁盘中断, 唤醒PB(I/O唤醒, 优先级高)
Switch()调度, PB上台, 返回用户态
- ⑥ 下一次时钟中断, PB响应, 唤醒PA, Switch()选中PA执行Sleep()系统调用下半部分

2. 时钟中断无响应.

